

08 QA Testing Plan

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

08 QA Testing Plan	4
title: "Cartridge & Cloud — QA Testing Plan" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" unity_test_framework: "1.6.0" application_version_reference: "0.0.26" documentary_baseline: "v0.6"	5
Cartridge & Cloud — QA Testing Plan	5
0. Propósito	5
0.1. Jerarquía de autoridad	5
0.2. Fotografía de estado	6
0.3. Evidencia operativa observada	6
1. Política de calidad	7
1.1. Definición de calidad	7
1.2. Principios	7
1.3. Shift-left y shift-right	7
1.4. Independencia pragmática	8
2. Alcance y exclusiones	8
2.1. Incluido	8
2.2. Excluido en la baseline actual	9
2.3. No objetivos	9
3. Riesgos de calidad	9
3.1. Riesgos críticos	9
3.2. Riesgos altos	10
3.3. Riesgos de experiencia	10
3.4. Priorización basada en riesgo	10
4. Organización y responsabilidades	10
4.1. Roles	10
4.2. Responsabilidad del desarrollador	11
4.3. Responsabilidad de QA	11
4.4. Responsabilidad de producción	11
5. Modelo de pruebas	11
5.1. Pirámide	11
5.2. Niveles	12
5.3. Tipos de ejecución	12
5.4. Técnicas de diseño de casos	13
5.5. Oráculos de prueba	15
5.6. Auditoría de estado antes y después	15
5.7. Fault injection	15
5.8. Calidad de los propios tests	16
5.9. Cobertura	16
6. Entornos y configuraciones	16
6.1. Entorno de desarrollo	16
6.2. Entorno de build	17
6.3. Resoluciones y modos	17
6.4. Hardware	18
6.5. Datos persistentes y aislamiento	18
7. Gestión de datos de prueba	18
7.1. Principios	18
7.2. IDs	18
7.3. Seeds y tiempo	19
7.4. Builders y fixtures	19
7.5. Golden datasets	19
8. EditMode	19
8.1. Objetivo	19

8.2. Cobertura obligatoria	19
8.3. Estructura Arrange-Act-Assert	20
8.4. Pruebas negativas	20
8.5. Atomicidad	20
8.6. Idempotencia	20
9. PlayMode e integración Unity	21
9.1. Objetivo	21
9.2. Reglas de estabilidad	21
9.3. Scene smoke tests	21
9.4. StoreInitial	22
10. Pruebas manuales	22
10.1. Objetivo	22
10.2. Sesión dirigida	22
10.3. Sesión exploratoria	22
10.4. Heurísticas	23
10.5. Evidencia visual	23
11. Regresión	23
11.1. Política	23
11.2. Núcleo de regresión	24
11.3. Selección basada en impacto	24
11.4. Tests obsoletos	24
12. Estrategia por sistema	24
12.1. Bootstrap, MainMenu y escenas	24
12.2. Input, movimiento y cámara	25
12.3. Grid, placement y acceso	25
12.4. Productos e inventario	26
12.5. Proveedores, pedidos y recepción	26
12.6. Displays y reposición	27
12.7. Clientes y spawning	27
12.8. Shopping, reservas y carrito	27
12.9. Cola y checkout	28
12.10. Día y cierre	28
12.11. Economía	28
12.12. Persistencia	29
12.13. UI, tutorial y accesibilidad	29
12.14. Arte, audio, VFX y StoreInitial	30
13. Golden Path	30
13.1. Objetivo	31
13.2. Recorrido obligatorio	31
13.3. Fallos mínimos dentro del Golden Path	31
13.4. Golden Path de siete días	31
14. Build y Player.log	32
14.1. Cuándo es obligatoria	32
14.2. Preflight	32
14.3. Smoke externo	32
14.4. Revisión de log	32
14.5. Artefactos	33
15. Rendimiento y estabilidad	33
15.1. Escenarios	33
15.2. Métricas	33
15.3. Procedimiento	34
15.4. Objetivos	34
15.5. Soak	34
16. Compatibilidad PC	34
16.1. Baseline inicial	34
16.2. Matriz progresiva	35
16.3. Instalación y actualización	35
17. Accesibilidad y localización	35
17.1. Accesibilidad	35

17.2. Localización	35
17.3. Pseudolocalización	36
18. Balance	36
18.1. Objetivo	36
18.2. Campañas	36
18.3. Variables	36
18.4. Métricas	36
18.5. Regla	37
19. Gestión de defectos	37
19.1. Estados	37
19.2. Severidad S0-S4	37
19.3. Prioridad	37
19.4. Informe mínimo	38
19.5. Verificación	38
19.6. Triage	38
19.7. Análisis de causa raíz	38
19.8. Seguridad de archivos y privacidad	39
19.9. Diagnóstico	39
20. Estados de resultados	39
21. Entrada, suspensión, reanudación y salida	40
21.1. Criterios de entrada	40
21.2. Suspensión	40
21.3. Reanudación	40
21.4. Salida de sprint	41
21.5. Salida de hito	41
22. Gates de Sprint 16	41
22.1. Estado final documentado	41
22.2. Criterios automáticos	41
22.3. Criterios manuales	42
22.4. Condición de cierre	42
22.5. Signoff de cierre	42
23. Gates de Sprint 17	42
23.1. Entrada	42
23.2. Campañas	43
23.3. Salida	43
24. Gate H6 — Vertical Slice Approved	44
24.1. Paquete de evidencia	44
24.2. Decisión	44
25. QA de fases posteriores	44
25.1. Empleados	44
25.2. Investigación	45
25.3. Puestos informáticos	45
25.4. Comercio online	45
25.5. Publishing y desarrollo	46
26. Demo, Alpha, Beta, RC y lanzamiento	46
26.1. Demo	46
26.2. Alpha	46
26.3. Beta	46
26.4. Release Candidate	46
26.5. Lanzamiento	47
27. QA externo y playtest	47
27.1. Orden	47
27.2. Protocolo	47
27.3. Métricas	47
28. Automatización continua	48
28.1. Objetivo	48
28.2. Pipeline recomendado	48
28.3. Requisitos	48
28.4. Flaky tests	48

28.5. Cadencia de ejecución	48
Durante implementación	48
Antes de integrar	48
Antes de cerrar sprint	49
Antes de hito	49
Tras hotfix futuro	49
28.6. Criterios para automatizar	49
29. Evidencias y reporting	50
29.1. Test Plan de sprint	50
29.2. Acceptance Matrix	50
29.3. QA Execution Record	50
29.4. Manual Validation Record	50
29.5. Build Execution Record	50
29.6. Informe de hito	51
30. Relación con 08_QA_Testing_Matrix.xlsx	51
30.1. Función	51
30.2. Hojas recomendadas	51
30.3. Campos mínimos de caso	51
30.4. Campos de ejecución	52
30.5. Identificadores	52
31. Definition of Ready para QA	53
32. Definition of Done de QA	53
33. Catálogo mínimo de campañas	53
33.1. Checklist maestra de ejecución	54
33.2. Auditoría cruzada de invariantes	56
33.3. Muestreo y repetición	56
33.4. Criterios de confianza	56
33.5. Recomendación final de QA	57
34. Mantenimiento del plan	57
Anexo A. Plantillas	57
A.1. Test Plan	58
A.2. QA Execution Record	58
A.3. Defecto	58
A.4. Build Record	58
Anexo B. Prácticas históricas conservadas	59
Anexo C. Prácticas reforzadas	59
Anexo E. Ruta de almacenamiento	59
Actualización QA W0	60
Estado operativo vigente tras W0	60
Pruebas de caracterización previas a las olas	60
Reglas de ejecución y evidencia	60
Regla de cierre y no propagación	60
Regresión adicional S17-MOD-002...010	61

08 QA Testing Plan

Metadatos normativos v2.0
document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

Nota de coherencia v1.1: las reglas de tiempo, pausa, apertura, cierre, liquidación y autosave diario se interpretan conforme a ADR-0086 y ADR-0087. Los estados de sprint o baseline anteriores se mantienen únicamente como fotografía histórica cuando una actualización posterior los sustituye.

title: "Cartridge & Cloud — QA Testing Plan" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" unity_test_framework: "1.6.0" application_version_reference: "0.0.26" documentary_baseline: "v0.6"

Cartridge & Cloud — QA Testing Plan

0. Propósito

Este documento define la estrategia de calidad de *Cartridge & Cloud* desde la baseline actual hasta el vertical slice, los hitos de producción posteriores y el lanzamiento en PC/Steam.

Su objetivo no es enumerar cada caso de prueba individual. Esa función corresponderá a la futura 08_QA_Testing_Matrix.xlsx. Este plan establece:

- qué significa calidad para el proyecto;
- qué riesgos reciben prioridad;
- qué niveles de prueba se utilizan;
- qué responsabilidades tiene cada nivel;
- qué entornos y configuraciones deben validarse;
- cómo se diseñan, ejecutan y documentan las pruebas;
- qué evidencias son obligatorias;
- cómo se gestionan defectos y regresiones;
- qué condiciones abren, suspenden, reanudan y cierran una campaña de QA;
- qué gates aplican a Sprint 16, Sprint 17 y H6;
- qué requisitos de QA se añaden en Alpha, Beta, Release Candidate y lanzamiento.

El plan convierte las reglas funcionales y técnicas en una política operativa de verificación. No sustituye:

- al GDD, que define las reglas del juego;
- a la Vertical Slice Specification, que define la aceptación del slice;
- al TDD, que define la arquitectura y las fronteras técnicas;
- al Modelo de Datos, que define ownership e invariantes;
- al UX Flow, que define recorridos observables;
- al Roadmap, que define gates y orden de ejecución;
- a la matriz QA, que contendrá los casos, variantes y resultados trazables.

0.1. Jerarquía de autoridad

La consolidación aplica el siguiente orden:

1. 00_Enfoque_y_Alcance.md
2. 01_Game_Design_Document.md
3. 02_Vertical_Slice_Specification.md
4. 03_Technical_Design_Document.md
5. 04_Modelo_de_Datos.md
6. 05_UX_Flow.md
7. 06_Production_Roadmap_y_Sprint_Plan.md
8. Cartridge_And_Cloud_QA_Testing_Plan_v0.6.md

9. registros QA de Sprint 16 y Sprint 16 Phase 1
10. registros operativos de Sprints 1-15
11. Cartridge_And_Cloud_QA_Testing_Plan_v0.5.md
12. Cartridge_And_Cloud_QA_Testing_Plan_v0.4.md
13. Cartridge_And_Cloud_QA_Testing_Plan_v0.3.md

Cuando exista una contradicción:

- prevalecen los documentos consolidados de mayor autoridad;
- la evidencia ejecutada prevalece sobre un plan antiguo;
- un PASS automatizado no sustituye una aceptación visual o manual pendiente;
- una prueba histórica basada en un alcance sustituido no se conserva como requisito vigente;
- la tienda de referencia es `StoreInitial.unity`, aproximadamente 10 × 15 m, con grid lógico 20 × 30 a 0,5 m;
- el catálogo representativo inicial es el aprobado por los documentos vigentes, no los antiguos mínimos de 12 productos y 6 familias;
- la severidad se expresa como S0–S4;
- ningún gate cierra con defectos S0 o S1 abiertos;
- los S2 requieren corrección o aceptación formal con riesgo, owner y plan;
- la build y el `Player.log` forman parte de la evidencia cuando el gate los exige.

0.2. Fotografía de estado

La baseline documental de referencia registra:

Elemento	Estado de referencia
Sprints 0-15	CLOSED / PASS
Sprint 16	COMPLETED / PASS
Sprint 17	APPROVED / IMPLEMENTED / VALIDATED
EditMode	1215 PASS
PlayMode	70 PASS
HISTORICAL EVIDENCE — infraestructura automatizada retirada:	Total automatizado
Aplicación	0.0.26
Plataforma de build	Windows x64
Escena objetivo	StoreInitial.unity

Los recuentos y SHA históricos deben registrarse de nuevo para cada baseline. No se deben copiar como si fueran resultados actuales después de modificar código, assets, escenas, packages o Project Settings.

0.3. Evidencia operativa observada

El corpus de desarrollo contiene **91 archivos Markdown de QA** bajo los registros operativos de Sprints 1-16. Incluye matrices de aceptación, registros de ejecución, planes, checklists, validaciones manuales, builds e incidencias.

La práctica real demuestra una disciplina útil:

- crecimiento progresivo de regresión;
- recuentos esperados y reales;
- aceptación por criterio;
- validación manual por sistema;
- build Windows x64 en gates;
- registro de incidentes de integración;
- recomendación explícita de apertura del siguiente sprint.

También revela puntos a mejorar:

- algunos registros dependen de resultados comunicados sin export del Test Runner;
- algunas builds no conservan ruta, tamaño, hash o log exportado;
- algunos planes recientes se redujeron demasiado y perdieron detalle estratégico;
- no todos los registros identifican hardware, resolución o configuración;
- el Player .log no se revisó de forma uniforme en los primeros sprints;
- un estado PASS debe distinguir prueba ejecutada, evidencia disponible y aceptación final.

Este plan conserva las prácticas útiles y convierte esas carencias en requisitos formales.

1. Política de calidad

1.1. Definición de calidad

La calidad de *Cartridge & Cloud* es la capacidad de ofrecer una experiencia:

- correcta respecto al diseño;
- coherente entre datos, lógica, escena y UI;
- resistente a entradas inválidas;
- recuperable ante fallos previstos;
- persistente sin pérdida ni duplicación;
- comprensible para el jugador;
- estable fuera del Editor;
- suficientemente fluida en el hardware objetivo;
- accesible según el alcance aprobado;
- reproducible mediante build, versión, SHA y evidencia.

Un sistema no se considera de calidad solo porque no lanza excepciones. Debe preservar invariantes, comunicar su estado, fallar sin mutaciones parciales y encajar en el recorrido completo.

1.2. Principios

1. **Prevenir antes que detectar.** Contratos, tipos, ownership y validadores reducen defectos.
2. **Probar en la capa correcta.** Las reglas puras no necesitan escenas; la integración visual sí.
3. **Automatizar lo determinista.** Invariantes, transiciones y serialización deben repetirse sin intervención manual.
4. **Validar manualmente lo perceptivo.** Legibilidad, sensación, composición, accesibilidad y coherencia visual requieren observación.
5. **Probar la build real.** El Editor no representa por completo inicialización, archivos, rendimiento, resolución ni salida.
6. **Proteger el histórico.** Toda capacidad aceptada entra en regresión.
7. **No ocultar incertidumbre.** PENDING, NOT RUN, BLOCKED y INCONCLUSIVE son estados legítimos; no deben convertirse en PASS por conveniencia.
8. **Registrar el contexto.** Un resultado sin versión, SHA, entorno o configuración tiene valor limitado.
9. **Tratar datos y dinero como críticos.** Pérdida, duplicación o desincronización tienen máxima prioridad.
10. **Cerrar por evidencia.** El volumen de código o de tests no sustituye el cumplimiento de los criterios de aceptación.

1.3. Shift-left y shift-right

Shift-left La calidad comienza antes de implementar:

- revisión de requisitos;
- criterios de aceptación;
- diseño de invariantes;

- estrategia de IDs;
- ownership del estado;
- análisis de riesgos;
- testabilidad;
- datos de prueba;
- compatibilidad de schema;
- límites de escena y composición.

Shift-right Después de integrar se valida:

- build;
- rendimiento;
- logs;
- sesiones prolongadas;
- playtests;
- compatibilidad de configuración;
- recuperación de archivos;
- telemetría o métricas autorizadas;
- incidencias reales de distribución cuando llegue esa fase.

1.4. Independencia pragmática

El proyecto es principalmente individual. La misma persona puede diseñar, implementar y probar. Para reducir sesgo de confirmación se exige:

- ejecutar una checklist escrita, no confiar en memoria;
 - probar desde una build limpia;
 - alternar pruebas dirigidas y exploratorias;
 - registrar resultados antes de corregirlos;
 - usar datos y seeds conocidos;
 - repetir recorridos después de reiniciar Unity y el equipo cuando sea relevante;
 - solicitar playtest externo en gates de experiencia;
 - separar la sesión de implementación de la sesión de aceptación cuando sea posible.
-

2. Alcance y exclusiones

2.1. Incluido

El plan cubre:

- compilación y dependencias;
- assemblies y paquetes;
- escenas y lifecycle;
- input, movimiento y cámara;
- construcción, grid, ocupación y acceso;
- productos, proveedores, pedidos y recepción;
- inventario, displays y reposición;
- clientes, navegación, preferencias y paciencia;
- shopping, reservas lógicas y carrito;
- cola, checkout y transacciones;
- ciclo diario y cierre;
- economía, precios, costes y ledger;
- slots, guardado, backup, recuperación y migración;
- HUD, Operations, tutorial, accesibilidad y localización;
- arte representativo, audio, VFX y autoría de StoreInitial;
- rendimiento, estabilidad y compatibilidad PC;
- build Windows x64;

- Golden Path;
- playtest y QA externo;
- hitos posteriores aprobados.

2.2. Excluido en la baseline actual

Hasta que se aprueben sus fases, no se exige QA funcional completo de:

- empleados;
- investigación;
- puestos informáticos;
- comercio online;
- publishing;
- desarrollo interno;
- plataforma digital;
- infraestructura avanzada;
- competidores y mercado completo;
- Steamworks, logros o Steam Cloud;
- consola;
- multijugador;
- modding.

Sí se exige que la arquitectura actual no bloquee su evolución y que cualquier placeholder o botón diferido permanezca oculto o claramente no operativo.

2.3. No objetivos

Este plan no pretende:

- demostrar ausencia absoluta de defectos;
- sustituir revisión de código;
- convertir cada detalle visual en test automatizado;
- imponer cobertura porcentual sin relación con el riesgo;
- mantener tests obsoletos solo para conservar un número alto;
- fijar hardware mínimo antes de medir;
- asumir que una sesión manual sustituye regresión;
- probar contenido futuro no comprometido.

3. Riesgos de calidad

3.1. Riesgos críticos

Riesgo	Ejemplo	Severidad potencial
corrupción de guardado	primario y backup inutilizables	S0
pérdida o duplicación de stock	transferencia parcial o recepción repetida	S0/S1
dinero incorrecto	venta o coste aplicado dos veces	S0/S1
progreso bloqueado	no se puede abrir, cerrar, guardar o continuar	S1
checkout no atómico	consume reserva sin registrar venta	S1
cierre bloqueado	cliente, cola o reserva no se resuelve	S1
acceso imposible	entrada o anchor obligatorio bloqueado	S1
escena duplicada	shell procedural y autorado coexisten	S1/S2
click-through de UI	botón mueve o coloca en el mundo	S2, S1 si bloquea S16
desincronización visual	display muestra stock inexistente	S1/S2

Riesgo	Ejemplo	Severidad potencial
migración inválida	schema anterior carga con estado incoherente	S0/S1
build no reproducible	funciona solo en Editor	S1

3.2. Riesgos altos

- rutas de cliente inestables;
- reservas no liberadas;
- cola no FIFO;
- doble confirmación por input repetido;
- placement que cobra antes de validar;
- retirada que destruye stock;
- pedido que cambia de estado dos veces;
- backup que sustituye al primario válido;
- UI sin estados de error;
- localización truncada en acciones críticas;
- allocations o stutter durante el flujo principal;
- dependencias por nombres de GameObject;
- pérdida de referencias al cambiar assets;
- configuración distinta entre Editor y Player.

3.3. Riesgos de experiencia

- el jugador no comprende dónde está el stock;
- confunde asignación con reposición;
- no entiende por qué un placement es inválido;
- no percibe paciencia o abandono;
- no encuentra la apertura o el cierre;
- no confía en el guardado;
- una señal depende solo de color o sonido;
- la cámara oculta objetos clave;
- la tienda representativa no comunica sus zonas;
- Results no explica beneficio, costes o problemas.

3.4. Priorización basada en riesgo

La prioridad de pruebas se determina por:

$\text{Riesgo} = \text{probabilidad} \times \text{impacto} \times \text{dificultad de detección tardía}$

No es necesario convertir la fórmula en un número exacto. Sirve para decidir:

- qué automatizar primero;
- qué ejecutar en cada commit;
- qué incluir en smoke;
- qué reservar para regresión completa;
- qué exige build;
- qué requiere playtest externo.

4. Organización y responsabilidades

4.1. Roles

Una persona puede asumir varios roles, pero las responsabilidades deben distinguirse.

Rol	Responsabilidad
Product/Design	define reglas, valor y aceptación
Development	implementa, instrumenta y corrige
QA	diseña pruebas, ejecuta, registra y recomienda gate
Art/Scene	garantiza escala, referencias, colliders y legibilidad
Production	decide prioridad, alcance y cierre
Build owner	genera, identifica y archiva build
Playtest observer	observa sin dirigir innecesariamente

4.2. Responsabilidad del desarrollador

- no entregar código que no compile;
- ejecutar tests dirigidos;
- documentar cambios de contrato;
- añadir pruebas para defectos reproducibles;
- no modificar tests solo para hacerlos pasar sin corregir el problema;
- indicar riesgos y áreas no probadas;
- conservar logs y datos necesarios.

4.3. Responsabilidad de QA

- derivar pruebas de requisitos e invariantes;
- evitar duplicación inútil;
- identificar huecos de cobertura;
- mantener datos y escenarios;
- registrar ambiente y resultado;
- separar defecto de diseño, código, datos, asset o documentación;
- recomendar PASS, PASS WITH ACCEPTED DEBT o FAIL;
- no aceptar por presión de calendario.

4.4. Responsabilidad de producción

- no abrir un sprint sin Definition of Ready;
- proporcionar tiempo para regresión;
- impedir expansión de alcance durante estabilización;
- resolver prioridad de defectos;
- aceptar deuda de forma explícita;
- proteger el gate de build y evidencia.

5. Modelo de pruebas

5.1. Pirámide

Playtest / QA externo
 Build / Golden Path / E2E
 Scene integration / PlayMode
 Application integration / EditMode
 Domain unit tests / validators / pure data

La mayor parte de las reglas deterministas debe vivir abajo. La parte superior es más costosa, menos estable y más cercana a la experiencia real.

5.2. Niveles

Nivel	Objetivo	Herramienta principal
estático	estructura, referencias y convenciones	validadores, compilación, scripts
unitario	value objects, reglas e invariantes	EditMode
integración	servicios y agregados	EditMode
componente Unity	MonoBehaviours, assets y lifecycle	PlayMode
escena	composición, referencias, navegación y UI	PlayMode + manual
funcional	recorrido de capacidad	manual o automatización dirigida
end-to-end	Golden Path	build externa
regresión	proteger capacidades cerradas	suite completa
exploratorio	descubrir riesgos no previstos	sesión manual
balance	ritmo y economía	simulaciones + playtest
rendimiento	frame, memoria y cargas	Profiler + build
compatibilidad	resoluciones, modos y rutas	matriz PC
accesibilidad	operación y legibilidad	manual + pruebas de UI
recuperación	fallos de archivo y estado	EditMode + build

5.3. Tipos de ejecución

Commit smoke Conjunto rápido para detectar:

- compilación;
- assemblies;
- escenas;
- servicios críticos;
- tests del cambio.

Targeted suite Pruebas específicas del sistema modificado.

Daily regression Suite automatizada completa o subconjunto estable cuando el tiempo lo requiera, sin sustituir el gate final.

Sprint gate

- suite completa;
- aceptación;
- manual;
- build si corresponde;
- logs;
- documentación.

Milestone gate Añade:

- Golden Path;
- rendimiento;
- compatibilidad;
- playtest;
- localización;
- recovery;
- evidence package completo.

5.4. Técnicas de diseño de casos

La matriz de pruebas debe utilizar técnicas explícitas. No basta con crear un caso por botón o por método.

Particiones de equivalencia Dividir entradas en grupos que deberían comportarse igual:

- cantidad válida, cero, negativa y por encima de capacidad;
- saldo suficiente, exacto e insuficiente;
- ID válido, duplicado, vacío y desconocido;
- posición válida, fuera de límites, ocupada y bloqueante;
- archivo válido, truncado, corrupto y de schema incompatible;
- estado permitido e incompatible.

Se selecciona al menos un representante de cada partición y se amplía cuando el riesgo es alto.

Valores límite Probar los bordes y sus vecinos:

mínimo - 1
mínimo
mínimo + 1
máximo - 1
máximo
máximo + 1

Aplicaciones:

- cantidades;
- capacidad;
- precio;
- saldo;
- paciencia;
- tiempo de apertura y cierre;
- celdas de grid;
- zoom;
- slots;
- versiones de schema;
- límites de clientes.

Tablas de decisión Adecuadas para operaciones con varias condiciones. Ejemplo de placement:

Dentro de límites	Libre	Acceso válido	Dinero	Resultado
sí	sí	sí	sí	confirmar
no	—	—	—	rechazar
sí	no	—	—	rechazar
sí	sí	no	—	rechazar
sí	sí	sí	no	rechazar

La tabla evita olvidar combinaciones y permite decidir cuáles son imposibles o redundantes.

Transiciones de estado Cada máquina de estados debe probar:

- transición válida;
- transición inválida;
- repetición;
- entrada y salida;
- acción durante transición;

- recuperación;
- persistencia.

Se aplica a:

- pedidos;
- entregas;
- clientes;
- reservas;
- cola;
- checkout;
- jornada;
- guardado;
- tutorial;
- navegación de escenas.

Pairwise Para combinaciones de menor riesgo —resolución, idioma, modo de pantalla, estado de tutorial, slot y método de entrada— se puede usar cobertura pairwise en lugar de probar el producto cartesiano completo. Las combinaciones críticas se mantienen explícitas.

Pruebas basadas en propiedades Las propiedades son invariantes aplicables a muchos datos generados:

- las cantidades nunca son negativas;
- transferir conserva unidades;
- reservar reduce available y aumenta reserved en la misma cantidad;
- cancelar restaura el estado anterior;
- serializar y deserializar conserva equivalencia;
- aplicar dos veces una operación idempotente no duplica efectos;
- el ledger explica cada cambio de saldo;
- ninguna cola contiene dos veces la misma entidad.

Cuando se utilicen datos aleatorios, se registra la seed y se limita el dominio para permitir reproducción.

Pruebas metamórficas Útiles cuando el resultado exacto es complejo, pero se conoce una relación:

- aumentar stock sin cambiar demanda no puede reducir disponibilidad inicial;
- rotar cuatro veces devuelve la orientación original;
- guardar, cargar y guardar de nuevo produce un estado equivalente;
- cambiar un texto localizado no altera lógica;
- duplicar la duración de una simulación estable no debe producir crecimiento de memoria lineal no explicado;
- retirar y recolocar el mismo mueble en la misma posición conserva ocupación.

Error guessing La experiencia histórica orienta pruebas específicas:

- doble clic;
- click-through;
- referencia serializada perdida;
- asset con pivot o escala incorrectos;
- dependencia de un nombre;
- escena cargada directamente;
- paquete con API de NUnit incompatible;
- build con escena ausente;
- save parcialmente escrito;
- fallback procedural activo junto a contenido autorado;
- cierre mientras un cliente cambia de estado.

5.5. Oráculos de prueba

Un oráculo determina qué es correcto. Debe identificarse para evitar aserciones ambiguas.

Área	Oráculo principal
reglas	GDD y Vertical Slice Specification
arquitectura	TDD
ownership e invariantes	Modelo de Datos
recorrido visible	UX Flow
gate	Roadmap
estado persistido	snapshot validado y contratos de restauración
escena	referencias explícitas y target visual aprobado
economía	ledger y saldo derivado
inventario	suma por ubicaciones y reservas
build	versión/SHA/artefacto/log identificados

No se debe usar la representación visual como único oráculo del estado. Tampoco el modelo lógico puede justificar una representación engañosa. Para operaciones críticas se comparan ambos.

5.6. Auditoría de estado antes y después

Para una operación mutante se recomienda capturar una proyección canónica antes y después:

```
StateDigest
├─ day/state/time
├─ money/ledger count
├─ inventory by location/product
├─ reservations
├─ orders/deliveries
├─ displays/assignments
├─ queue/transactions
├─ placements/occupancy
└─ schema/version
```

El digest no reemplaza al snapshot. Es una vista compacta para detectar efectos laterales no esperados y comparar save/load, cancelación y fallos.

5.7. Fault injection

Los fallos difíciles deben introducirse en seams controlados, sin corromper el entorno real.

Ejemplos:

- repositorio devuelve error de escritura;
- archivo temporal no puede promoverse;
- catálogo no contiene un ID;
- referencia de escena ausente;
- destino de navegación inválido;
- servicio de audio no disponible;
- operación intenta confirmar dos veces;
- excepción entre preflight y commit simulada;
- backup válido con primario truncado;
- schema futuro no soportado.

La prueba debe comprobar:

- mensaje y código de error;
- ausencia de mutación parcial;

- cleanup;
- posibilidad de reintento;
- logs;
- conservación del primario o backup;
- estado de UI.

5.8. Calidad de los propios tests

Un test es código de producción interna. Debe ser:

- legible;
- determinista;
- independiente;
- rápido para su nivel;
- preciso al fallar;
- resistente a refactors legítimos;
- compatible con la versión real del framework.

Antipatrones:

- sleeps fijos;
- orden implícito;
- asserts genéricos sin contexto;
- acceso global sin cleanup;
- varios comportamientos no relacionados en un test;
- usar el mismo código de producción para calcular el esperado;
- mocks que replican toda la implementación;
- depender de nombres de jerarquía no contractuales;
- ignorar warnings de compilación;
- actualizar snapshots dorados sin revisión.

5.9. Cobertura

No se establece un porcentaje universal de líneas. Se usa cobertura de requisitos, riesgos, invariantes, estados y recorridos.

Cada capacidad debe responder:

- ¿qué requisito cubre?;
- ¿qué riesgo reduce?;
- ¿qué caminos positivos y negativos existen?;
- ¿qué estados iniciales y finales cubre?;
- ¿qué ocurre al cancelar?;
- ¿qué ocurre al repetir?;
- ¿qué ocurre al guardar/cargar?;
- ¿qué evidencia demuestra el resultado?;

Los informes de cobertura de código pueden ayudar a encontrar zonas no ejecutadas, pero no son un criterio de aceptación por sí solos.

6. Entornos y configuraciones

6.1. Entorno de desarrollo

Registrar:

- sistema operativo;
- versión exacta de Unity;

- versión de URP;
- packages relevantes;
- branch y SHA;
- versión de aplicación;
- estado de working copy;
- backend y arquitectura;
- configuración de build;
- resolución y modo de pantalla;
- idioma;
- dispositivo de entrada;
- ruta de datos persistentes cuando sea relevante.

6.2. Entorno de build

Baseline inicial:

- Windows x64;
- Development Build para diagnóstico durante sprints;
- build limpia desde el flujo aprobado;
- entrada por Bootstrap;
- StoreInitial en la lista de escenas cuando su integración esté aprobada;
- ejecución fuera del Editor.

La build candidata de hito debe registrar:

- nombre de carpeta o artefacto;
- versión;
- SHA;
- fecha y hora;
- tamaño;
- hash del paquete o ejecutable cuando sea posible;
- flags de desarrollo;
- arquitectura;
- ruta de Player.log archivada;
- resultado de antivirus o firma cuando llegue Release Candidate.

6.3. Resoluciones y modos

Como mínimo para el vertical slice:

- 1280 × 720;
- 1920 × 1080 de referencia;
- 2560 × 1440;
- ventana;
- borderless;
- fullscreen si está disponible.

Se validan:

- escala de UI;
- truncamiento;
- tooltips;
- modales;
- puntero;
- cámara;
- relación de aspecto;
- persistencia de opciones.

6.4. Hardware

Hasta aprobar requisitos mínimos, cada prueba de rendimiento debe registrar el equipo real:

- CPU;
- GPU;
- RAM;
- almacenamiento;
- resolución;
- calidad gráfica;
- VSync;
- límite de FPS;
- estado de batería/energía en portátil;
- drivers cuando sea material.

El objetivo histórico de 60 FPS a 1920 × 1080 con tienda equipada y hasta 8 clientes es una referencia provisional, no un requisito mínimo publicado hasta medir y aprobar perfiles.

6.5. Datos persistentes y aislamiento

Las campañas deben usar:

- carpeta limpia para nueva partida;
- copia controlada para corrupción;
- slots independientes;
- backup conocido;
- permisos de escritura normales;
- rutas con caracteres no ASCII;
- nombres de usuario largos cuando sea viable;
- restauración del entorno después de la prueba.

No se deben ejecutar pruebas destructivas sobre el único guardado de desarrollo.

7. Gestión de datos de prueba

7.1. Principios

Los datos de prueba deben ser:

- pequeños;
- legibles;
- deterministas;
- versionados;
- representativos;
- independientes de assets accidentales;
- fáciles de restaurar.

7.2. IDs

- utilizar IDs estables explícitos;
- probar IDs duplicados, vacíos y desconocidos;
- no depender de nombres visibles;
- conservar IDs entre save/load;
- validar referencias cruzadas;
- probar desaparición de una definición requerida.

7.3. Seeds y tiempo

Cuando exista aleatoriedad:

- permitir seed conocida;
- registrar seed en defectos;
- separar tests deterministas de pruebas estadísticas;
- controlar reloj lógico;
- evitar depender de tiempo real;
- cubrir límites de día, cierre y semana.

7.4. Builders y fixtures

Los tests deben usar builders con defaults válidos y overrides explícitos.

Un builder no debe:

- ocultar valores críticos;
- crear estado imposible;
- mutar singletons globales sin cleanup;
- acoplarse a jerarquías visuales;
- compartir estado mutable entre tests.

7.5. Golden datasets

Mantener conjuntos aprobados para:

- catálogo mínimo representativo;
 - mobiliario;
 - proveedores;
 - perfiles de cliente;
 - economía inicial;
 - snapshot schema actual;
 - snapshot schema anterior soportado;
 - guardado corrupto controlado;
 - escena StoreInitial.
-

8. EditMode

8.1. Objetivo

Validar reglas puras, servicios de aplicación, serialización, repositorios y validadores sin el coste de una escena.

8.2. Cobertura obligatoria

- construcción y validación de value objects;
- equality y hashing;
- cantidades y dinero;
- IDs;
- transiciones de estado;
- precondiciones;
- atomicidad;
- conservación de unidades;
- idempotencia;
- orden de colas;
- reservas;

- cálculo de precios y ledger;
- snapshots;
- schema y migración;
- checksums;
- backup;
- validación de catálogos;
- dependencia de assemblies;
- lista de escenas;
- assets de authoring cuando puedan inspeccionarse sin PlayMode.

8.3. Estructura Arrange-Act-Assert

Cada test debe:

1. preparar un estado mínimo;
2. ejecutar una acción principal;
3. comprobar resultado y efectos laterales;
4. comprobar que no cambió lo que debía permanecer intacto;
5. liberar recursos.

8.4. Pruebas negativas

Cada servicio mutante debe cubrir:

- argumentos nulos o inválidos;
- cantidades cero y negativas;
- IDs desconocidos;
- estado incompatible;
- saldo insuficiente;
- capacidad insuficiente;
- reserva inexistente;
- confirmación repetida;
- operación tras cierre;
- excepción de repositorio simulada cuando sea viable.

8.5. Atomicidad

Patrón mínimo:

capturar estado antes
→ ejecutar operación inválida o fallo inyectado
→ comprobar resultado de error
→ comparar estado después con estado antes

Debe aplicarse a:

- transferencias;
- pedido;
- recepción;
- reposición;
- reserva;
- checkout;
- placement;
- retirada;
- movimiento de dinero;
- guardado.

8.6. Idempotencia

Probar doble ejecución en:

- recepción;
 - confirmación de checkout;
 - cierre;
 - autosave;
 - recuperación;
 - registro de escena;
 - carga de catálogo;
 - aplicación de migración.
-

9. PlayMode e integración Unity

9.1. Objetivo

Validar comportamiento que depende de:

- MonoBehaviour;
- escenas;
- serialización Unity;
- Input System;
- EventSystem;
- física;
- NavMesh;
- lifecycle;
- coroutines;
- animación;
- audio;
- composición.

9.2. Reglas de estabilidad

- esperar condiciones, no tiempos arbitrarios;
- usar timeout explícito;
- destruir objetos creados;
- restaurar `Time.timeScale`;
- no depender del orden de tests;
- cargar escenas de forma controlada;
- comprobar número de roots o contextos cuando sea contrato;
- evitar asserts incompatibles con la versión de NUnit del proyecto.

La incidencia de Sprint 8 con `Assert.Multiple` demuestra que la compatibilidad del framework de tests debe verificarse antes de distribuir paquetes de pruebas.

9.3. Scene smoke tests

Cada escena de producción debe comprobar:

- carga sin excepción;
- cámara y EventSystem según contrato;
- root de composición correcto;
- ausencia de roots duplicados;
- referencias obligatorias;
- transición de entrada y salida;
- contexto de input;
- cleanup.

9.4. StoreInitial

Automatizar cuando sea estable:

- existencia de escena;
- presencia de StoreInitialSceneContext;
- referencias obligatorias;
- una sola arquitectura visible;
- anchors y roots;
- zonas;
- relación básica con grid;
- colliders necesarios;
- ausencia de shell procedural simultáneo;
- carga nueva y restaurada;
- no duplicación de muebles dinámicos.

La aprobación de escala, composición, iluminación y legibilidad permanece manual.

10. Pruebas manuales

10.1. Objetivo

Cubrir aspectos que una aserción automática no representa bien:

- sensación de control;
- claridad;
- composición;
- feedback;
- accesibilidad;
- continuidad visual;
- audio;
- ritmo;
- descubrimiento;
- frustración;
- confianza en guardado.

10.2. Sesión dirigida

Utiliza pasos y resultados esperados. Es adecuada para acceptance y regresión.

Debe registrar:

- entorno;
- build;
- precondiciones;
- pasos;
- resultados;
- evidencia;
- incidencias;
- datos usados.

10.3. Sesión exploratoria

Se define por charter, por ejemplo:

Explorar BuildMode durante 45 minutos para descubrir formas de crear solapes, bloquear rutas, confirmar dos veces o dejar previews huérfanos.

Registrar:

- charter;
- duración;
- áreas cubiertas;
- variaciones;
- defectos;
- preguntas;
- riesgos no cubiertos.

10.4. Heurísticas

- cancelar en cada paso;
- repetir rápidamente;
- alternar UI y mundo;
- cambiar escena con un modal abierto;
- guardar en límites permitidos;
- cargar después de cada estado significativo;
- agotar recursos;
- llenar capacidades;
- usar valores mínimos y máximos;
- dejar procesos a medias;
- cerrar la tienda con agentes en cada estado;
- cambiar resolución e idioma con paneles abiertos;
- ejecutar varias jornadas sin reiniciar.

10.5. Evidencia visual

Las capturas deben mostrar:

- versión/build cuando sea posible;
- estado relevante;
- defecto completo, no solo un recorte ambiguo;
- escala o referencia espacial para problemas de escena;
- antes y después para correcciones visuales.

El vídeo es preferible para:

- input;
 - animación;
 - cámara;
 - rutas;
 - colas;
 - stutter;
 - flujo completo.
-

11. Regresión

11.1. Política

Toda capacidad CLOSED / PASS se incorpora a regresión proporcional a su riesgo.

Una modificación debe ejecutar:

- tests dirigidos del área;
- tests de dependencias directas;
- smoke de escena si toca composición;
- regresión completa antes del cierre.

11.2. Núcleo de regresión

- Bootstrap y scene flow;
- slots;
- input y contextos;
- movimiento y cámara;
- placement;
- acceso;
- inventario;
- proveedores;
- displays;
- clientes;
- shopping;
- checkout;
- día;
- economía;
- persistencia;
- UI;
- audio/VFX;
- assets representativos.

11.3. Selección basada en impacto

Cambio	Regresión mínima adicional
assembly/package	compilación, grafo, escenas, suite completa
escena	smoke, input, lifecycle, Golden Path afectado
datos	validadores, referencias, save/load
inventario	pedidos, displays, reservas, checkout, save
cliente	spawning, shopping, cola, cierre, rendimiento
UI	input gating, foco, localización, resolución
persistencia	todos los agregados y recovery
build profile	build limpia y Player.log
asset representativo	prefabs, colliders, LOD, escena y rendimiento

11.4. Tests obsoletos

Un test puede retirarse solo si:

- la regla fue sustituida formalmente;
- está duplicado sin valor;
- prueba implementación accidental en lugar de contrato;
- es flaky y se reemplaza por cobertura equivalente;
- se registra la razón.

No se modifica el expected para ocultar una regresión sin decisión de producto.

12. Estrategia por sistema

12.1. Bootstrap, MainMenu y escenas

Verificar:

- entrada única por Bootstrap;
- un solo ApplicationRoot;

- transición asíncrona;
- rechazo de solicitudes concurrentes;
- MainMenu funcional;
- selección de slot;
- carga de StoreInitial;
- retorno;
- Quit;
- direct play solo como fallback técnico autorizado;
- cleanup de contextos;
- fallos de escena recuperables.

Variaciones:

- diez loops consecutivos;
- multiclick rápido;
- reinicio de Unity;
- build externa;
- carga fallida simulada cuando exista seam;
- escena con referencia obligatoria ausente.

12.2. Input, movimiento y cámara

Verificar:

- exclusividad de contextos;
- clic sobre UI no llega al mundo;
- click-to-move;
- destino inválido;
- colisión;
- cancelación;
- órbita;
- zoom;
- límites;
- velocidad configurable;
- Escape;
- pausa;
- input durante loading;
- cierre de panel sin movimiento residual.

Pruebas de estrés:

- clics alternos UI/suelo;
- doble clic;
- mantener botón de cámara;
- cambiar contexto durante movimiento;
- abrir modal en BuildMode;
- cambiar resolución.

12.3. Grid, placement y acceso

Invariantes:

- snapping 0,5 m;
- rotación de 90°;
- bounds;
- no overlap;
- ocupación exacta;
- reserva de entrada;
- anchors alcanzables;
- rechazo sin mutación;

- coste solo tras éxito;
- retirada libera ocupación;
- movimiento es atómico;
- cancelación restaura estado;
- navegación se actualiza.

Variaciones:

- huellas pares e impares;
- bordes y esquinas;
- rotación que cambia acceso;
- dos placements simultáneos;
- retirada con stock;
- save/load;
- escena autorada y fallback;
- grid visual frente a superficie técnica.

12.4. Productos e inventario

Invariantes:

$on_hand = available + reserved$

y, para la vista física aprobada:

$stock\ total = recepción + almacén + exposición + reservas + otras\ ubicaciones\ explícitas$

Verificar:

- IDs y definiciones;
- cantidades no negativas;
- capacidad;
- compatibilidad;
- transferencias;
- conservación;
- atomicidad;
- ubicaciones;
- snapshots;
- representación visual;
- stock devuelto al retirar display;
- concurrencia lógica de reservas.

12.5. Proveedores, pedidos y recepción

Verificar:

- catálogo;
- disponibilidad;
- líneas;
- cantidad;
- coste;
- saldo;
- transición de estados;
- entrega única;
- recepción única;
- creación de cajas/unidades;
- destino;
- cancelación si existe;
- save/load en cada estado;
- fallo de capacidad;
- fallo de repositorio.

12.6. Displays y reposición

Verificar:

- definición y familia;
- slots;
- asignación no crea stock;
- compatibilidad;
- capacidad;
- reposición exacta;
- reposición parcial;
- origen insuficiente;
- reasignación;
- devolución de stock;
- retirada;
- sincronización visual;
- reload;
- múltiples displays con mismo producto.

12.7. Clientes y spawning

Verificar:

- perfiles;
- IDs únicos;
- seed;
- límite simultáneo;
- spawn válido;
- entrada;
- navegación;
- paciencia;
- estados;
- salida;
- fallo de ruta;
- cierre;
- cleanup;
- save policy de agentes transitorios.

12.8. Shopping, reservas y carrito

Verificar:

- intención;
- búsqueda;
- ranking;
- disponibilidad;
- reserva atómica;
- no sobreventa;
- carrito;
- abandono;
- liberación;
- expiración si existe;
- varias unidades;
- varios clientes;
- cierre;
- save/load;
- definición desaparecida.

12.9. Cola y checkout

Verificar:

- entrada FIFO;
- posiciones;
- estación;
- cliente actual;
- carrito válido;
- preflight;
- total;
- consumo de reservas;
- registro de venta;
- movimiento de dinero;
- idempotencia;
- fallo sin mutación;
- avance de cola;
- cliente que abandona;
- cierre con cola activa.

12.10. Día y cierre

Estados:

BeforeOpen → Open → Closing → Closed → Results → NextDay

Verificar:

- transiciones válidas e inválidas;
- spawn solo durante Open;
- aviso de cierre;
- bloqueo de nuevos clientes;
- resolución de agentes;
- liberación de reservas;
- resultado solo tras estabilizar;
- autosave;
- avance de día;
- semana;
- reloj lógico;
- pausa y velocidad;
- cierre repetido.

12.11. Economía

Verificar:

- dinero exacto;
- no usar floats para autoridad monetaria;
- costes de pedido;
- ingresos;
- precios;
- margen;
- ledger;
- referencia de transacción;
- saldo resultante;
- resultados;
- valores negativos válidos o rechazados según regla;
- impuestos cuando entren en alcance;
- save/load;
- doble aplicación.

12.12. Persistencia

Verificar:

- slot vacío;
- primera y segunda escritura;
- archivo temporal;
- promoción atómica;
- backup;
- checksum;
- timestamp;
- aislamiento;
- reemplazo;
- borrado y sidecars;
- corrupción primaria;
- corrupción doble;
- recuperación;
- reparación;
- schema soportado;
- schema futuro rechazado;
- ID desconocido;
- referencia cruzada inválida;
- captura y restauración en fases;
- equivalencia de snapshot;
- cleanup de temporales.

El snapshot integrado de referencia es IntegratedGameStateSnapshot; sus invariantes, schema y restauración deben verificarse como contrato de persistencia.

Todo agregado persistente nuevo exige:

1. captura;
2. serialización;
3. validación;
4. restauración;
5. rollback o error seguro;
6. round-trip;
7. integración con backup;
8. migración o política explícita.

12.13. UI, tutorial y accesibilidad

Verificar:

- slots New/Continue/Replace/Delete;
- foco;
- ratón;
- teclado;
- Escape;
- paneles;
- estados vacíos;
- loading;
- error;
- confirmación;
- feedback;
- tutorial;
- persistencia de progreso;
- escala;
- contraste;
- señales no cromáticas;

- velocidad de cámara;
- idioma ES/EN;
- truncamiento;
- navegación a 1280×720, 1920×1080 y 2560×1440;
- UI no propaga clics.

12.14. Arte, audio, VFX y StoreInitial

Assets

- importan sin error;
- escala y orientación;
- materiales;
- prefabs;
- colliders;
- pivots;
- LOD cuando exista;
- Missing Scripts;
- referencias;
- nombres y ubicación según estándar;
- licencias y procedencia cuando corresponda.

StoreInitial

- una sola arquitectura;
- suelo coincide con placement surface;
- muros bordean el espacio aprobado;
- puerta vertical y funcional;
- entrada no bloqueada;
- mobiliario inicial en posiciones aprobadas;
- almacén y recepción legibles;
- anchors accesibles;
- colliders correctos;
- wall occlusion;
- iluminación suficiente;
- no duplicación tras load;
- muebles dinámicos reaparecen;
- shell procedural desactivado en la ruta nueva.

Audio/VFX

- evento correcto;
- no doble reproducción;
- volumen y mezcla;
- pausa;
- scene transition;
- objeto destruido;
- fallback si falta clip;
- no spam;
- alternativa visual para avisos críticos;
- rendimiento.

13. Golden Path

13.1. Objetivo

Demostrar en una build externa que los sistemas principales forman una experiencia completa y persistente.

13.2. Recorrido obligatorio

Bootstrap
→ MainMenu
→ slot vacío
→ New Game
→ StoreInitial
→ movimiento y cámara
→ Operations
→ pedido
→ entrega
→ recepción
→ transferencia a almacén
→ asignación de producto
→ reposición
→ precio
→ abrir tienda
→ cliente compra
→ cola
→ checkout
→ dinero e inventario correctos
→ cerrar
→ Results
→ guardar/autoguardar
→ Next Day
→ volver a MainMenu
→ cargar slot
→ comprobar equivalencia

13.3. Fallos mínimos dentro del Golden Path

La campaña debe ejecutar también rutas de error controladas:

- placement inválido;
- saldo insuficiente;
- stock insuficiente;
- display lleno;
- UI sobre mundo;
- reserva liberada por abandono;
- cierre con cliente activo;
- guardado primario corrupto con backup válido;
- cancelación de una acción destructiva.

13.4. Golden Path de siete días

Antes de H6 debe completarse una secuencia suficiente para demostrar continuidad. La referencia histórica es una partida de siete días.

Durante los siete días registrar:

- saldo inicial y final;
- inventario por ubicación;
- pedidos y entregas;
- ventas;
- abandonos;
- resultados;
- guardados;

- cargas;
- errores;
- rendimiento observado;
- crecimiento de memoria si se mide.

No es necesario convertir cada día en el mismo guion. Se deben variar precios, pedidos, layout y carga de clientes para aumentar cobertura.

14. Build y Player.log

14.1. Cuándo es obligatoria

- cambios de scene flow;
- cambios de packages;
- cambios de build profile;
- cambios de persistencia;
- cierre de sprint funcional significativo;
- Sprint 16;
- Sprint 17;
- H6;
- demo, Alpha, Beta, RC y Release.

14.2. Preflight

- working copy conocida;
- compilación limpia;
- tests verdes;
- versión actualizada;
- escenas correctas;
- Bootstrap primero;
- carpeta de salida limpia;
- espacio suficiente;
- configuración registrada;
- Development Build según campaña;
- datos persistentes controlados.

14.3. Smoke externo

- ejecutable abre;
- no aparece diálogo de crash;
- Bootstrap;
- MainMenu;
- slots;
- StoreInitial;
- input;
- guardar;
- cargar;
- volver;
- Quit;
- no depende del Editor.

14.4. Revisión de log

Buscar:

- exceptions;

- stack traces;
- missing references;
- failed assertions;
- shader errors;
- asset load failures;
- save/recovery errors;
- warnings repetitivas;
- mensajes de fallback inesperado;
- duplicación de roots;
- rutas de archivo inválidas.

Clasificación:

- esperado y documentado;
- warning aceptable;
- defecto;
- ruido que debe eliminarse;
- información diagnóstica temporal.

Un log sin crash no es automáticamente limpio. Los warnings recurrentes pueden ocultar defectos y afectar rendimiento.

14.5. Artefactos

- ejecutable o paquete;
 - hash;
 - versión;
 - SHA;
 - `Player.log`;
 - test report;
 - checklist;
 - capturas;
 - known issues;
 - resultado.
-

15. Rendimiento y estabilidad

15.1. Escenarios

1. escena recién cargada;
2. tienda inicial vacía de clientes;
3. tienda equipada;
4. operación con carga representativa;
5. hasta 8 clientes simultáneos como referencia del slice;
6. apertura y cierre;
7. save/load;
8. varias jornadas consecutivas;
9. stress progresivo por encima de la carga esperada para encontrar degradación.

15.2. Métricas

- FPS medio;
- frame time;
- percentiles cuando la herramienta lo permita;
- CPU main thread;
- rendering;
- GPU;

- memoria;
- allocations;
- GC;
- tiempo de carga;
- tiempo de guardado;
- tiempo de carga de slot;
- spikes de navegación;
- UI rebuild;
- instanciación;
- audio/VFX.

15.3. Procedimiento

- usar build;
- registrar hardware;
- calentar escena;
- ejecutar ventana de medición conocida;
- repetir;
- comparar con baseline anterior;
- conservar captura o export;
- aislar cambios;
- no optimizar sin perfil.

15.4. Objetivos

El objetivo provisional es 60 FPS a 1080p en una configuración documentada. Hasta aprobar hardware mínimo, bloquean el gate:

- stutter persistente;
- caída grave respecto a baseline;
- crecimiento de memoria continuo;
- allocations recurrentes importantes en rutas estables;
- bloqueo visible de guardado/carga;
- tiempo de escena inaceptable;
- degradación acumulativa entre días.

15.5. Soak

Para Sprint 17 y posteriores:

- varias jornadas sin reiniciar;
- repetición de save/load;
- apertura/cierre reiterado;
- creación y retirada de objetos;
- entrada y salida de paneles;
- observación de memoria, logs y estado.

16. Compatibilidad PC

16.1. Baseline inicial

- Windows 10/11 mientras se mantenga el objetivo;
- teclado y ratón;
- rutas de usuario con caracteres no ASCII;
- permisos de usuario estándar;
- diferentes resoluciones;

- ventana/borderless/fullscreen;
- distintas tasas de refresco cuando sea posible.

16.2. Matriz progresiva

La matriz se amplía por hito:

Hito	Cobertura
Vertical Slice	equipo principal + resolución y modos básicos
Demo	varios equipos y GPUs representativas
Alpha	mínimos/recomendados provisionales
Beta	matriz de hardware y drivers priorizada
RC	requisitos publicados y configuraciones críticas

16.3. Instalación y actualización

En fases de distribución probar:

- instalación limpia;
- actualización sobre versión anterior;
- desinstalación;
- conservación o eliminación de saves según política;
- rutas largas;
- permisos;
- antivirus/falso positivo;
- rollback si aplica.

17. Accesibilidad y localización

17.1. Accesibilidad

Cobertura mínima:

- escala de UI;
- tamaño de texto;
- contraste;
- estados no dependientes solo de color;
- alternativas visuales a audio;
- reducción de movimiento cuando exista;
- sensibilidad y velocidad de cámara;
- pausa para lectura;
- navegación de teclado y foco;
- confirmaciones;
- lenguaje claro.

17.2. Localización

Para ES y EN:

- claves existentes;
- fallback;
- variables;
- plurales;
- moneda;

- fechas;
- truncamiento;
- solape;
- orden de palabras;
- mayúsculas;
- tooltips;
- modales;
- mensajes de error;
- tutorial;
- Results.

17.3. Pseudolocalización

Cuando la infraestructura lo permita:

- expansión de longitud;
 - caracteres acentuados;
 - delimitadores visibles;
 - detección de texto hardcoded;
 - layout RTL solo si se aprueba soporte futuro.
-

18. Balance

18.1. Objetivo

Comprobar que el sistema produce decisiones, no un único resultado inevitable.

18.2. Campañas

- 7 días para vertical slice;
- 30 días cuando exista progresión suficiente;
- 100 días para simulaciones de economía y extremos cuando sea razonable.

18.3. Variables

- capital inicial;
- costes;
- precios;
- demanda;
- frecuencia de clientes;
- paciencia;
- stock;
- tiempos de entrega;
- capacidades;
- duración del día;
- márgenes;
- gastos recurrentes;
- impuestos cuando entren en alcance.

18.4. Métricas

- insolvencia;
- margen;
- roturas de stock;
- abandonos;
- ventas;

- tiempo de recuperación;
- acumulación de dinero;
- decisiones dominantes;
- tiempo ocioso;
- carga operativa.

18.5. Regla

Los tests de balance no deben convertir valores actuales en contratos rígidos. Deben verificar rangos, tendencias e invariantes y dejar los valores exactos en configuración.

19. Gestión de defectos

19.1. Estados

Estado	Significado
NEW	registrado, pendiente de triage
CONFIRMED	reproducido
IN PROGRESS	en corrección
FIXED	cambio disponible, no verificado
VERIFIED	corrección validada
REOPENED	persiste o regresa
DEFERRED	aplazado con razón
ACCEPTED	deuda aceptada formalmente
DUPLICATE	ya registrado
NOT A BUG	comportamiento aprobado
CANNOT REPRODUCE	evidencia insuficiente, no equivale a cerrado definitivo

19.2. Severidad S0-S4

Severidad	Definición	Ejemplos
S0	pérdida grave, corrupción, seguridad o bloqueo total	save destruido, app no inicia
S1	flujo principal imposible o invariante crítica rota	stock duplicado, cierre imposible
S2	función importante degradada con workaround	UI mueve jugador, recuperación confusa
S3	fricción o defecto visible significativo	feedback inconsistente, layout menor
S4	cosmético o mejora menor	alineación, typo no crítico

La severidad depende del gate. En Sprint 16, muros, puerta o mobiliario representativo incorrectos son S1 porque bloquean el entregable, aunque en otra fase pudieran ser visuales.

19.3. Prioridad

La prioridad incorpora:

- severidad;
- frecuencia;
- alcance;
- gate;
- riesgo de datos;

- coste de corregir tarde;
- visibilidad para el jugador.

19.4. Informe mínimo

ID:

Título:

Build / versión / SHA:

Entorno:

Precondiciones:

Pasos:

Resultado actual:

Resultado esperado:

Frecuencia:

Severidad:

Evidencia:

Logs:

Datos/seed/slot:

Workaround:

Regresión relacionada:

19.5. Verificación

- ejecutar pasos originales;
- ejecutar variante negativa;
- ejecutar test de regresión;
- comprobar sistemas vecinos;
- revisar save/load si tocó estado;
- revisar build si el defecto era externo;
- cerrar solo con evidencia.

19.6. Triage

El triage se realiza:

- inmediatamente para S0/S1;
- antes del siguiente gate para S2;
- en la revisión periódica para S3/S4.

Preguntas:

1. ¿Es reproducible?
2. ¿Qué build y datos afecta?
3. ¿Es defecto, cambio de diseño o configuración?
4. ¿Amenaza datos o progresión?
5. ¿Existe workaround?
6. ¿Qué regresión requiere?
7. ¿Bloquea el sprint o hito?
8. ¿Debe informarse en known issues?

19.7. Análisis de causa raíz

Obligatorio para:

- S0;
- S1 repetido;
- corrupción;
- regresión de una capacidad cerrada;
- fallo de build tardío;
- defecto escapado a distribución;

- test flaky recurrente.

Formato recomendado:

síntoma

- causa inmediata
- causa sistémica
- por qué no se detectó antes
- corrección
- prevención
- cobertura añadida

No se utiliza el análisis para asignar culpa. Su objetivo es mejorar requisitos, diseño, instrumentación, tests o proceso.

19.8. Seguridad de archivos y privacidad

Aunque el juego sea offline en la baseline, QA debe proteger:

- guardados del usuario;
- configuraciones;
- logs;
- datos de playtest;
- rutas locales;
- credenciales futuras de servicios.

Reglas:

- no registrar secretos;
- no incluir datos personales innecesarios en logs;
- anonimizar playtests;
- no subir saves reales sin consentimiento;
- probar rutas de archivo sin ejecutar contenido arbitrario;
- tratar archivos externos como no confiables;
- validar tamaño y schema antes de cargar;
- conservar backups antes de migraciones destructivas.

19.9. Diagnóstico

Los mensajes diagnósticos deben ayudar a reproducir sin inundar el log.

Para operaciones críticas registrar, cuando proceda:

- operación;
- ID estable;
- slot;
- schema;
- resultado;
- código de error;
- transición;
- duración;
- recovery usado.

No registrar cada frame, movimiento o tick. Los logs repetitivos deben agregarse o limitarse.

20. Estados de resultados

Estado	Uso
PASS	resultado esperado demostrado
FAIL	resultado esperado no cumplido
BLOCKED	no puede ejecutarse por dependencia
NOT RUN	todavía no ejecutado
INCONCLUSIVE	evidencia insuficiente o resultado ambiguo
N/A	no aplica con justificación
SKIPPED	omitido deliberadamente con razón

Reglas:

- PENDING se normaliza a NOT RUN o BLOCKED en la matriz;
- no sumar NOT RUN como PASS;
- un test flaky es INCONCLUSIVE o FAIL, no PASS alterno;
- N/A requiere motivo;
- un criterio puede necesitar varias pruebas antes de ser PASS.

21. Entrada, suspensión, reanudación y salida

21.1. Criterios de entrada

Una campaña puede comenzar cuando:

- build o working copy está identificada;
- requisitos están aprobados;
- criterios existen;
- entorno está disponible;
- datos están preparados;
- bloqueos conocidos están registrados;
- el cambio compila;
- smoke mínimo pasa;
- la matriz contiene los casos aplicables.

21.2. Suspensión

Suspender si:

- no compila;
- la build no inicia;
- existe S0 que invalida resultados;
- el entorno está corrupto;
- faltan assets o datos críticos;
- la tasa de fallos impide continuar con valor;
- el requisito cambió;
- los resultados no pueden atribuirse a una versión.

21.3. Reanudación

- bloqueo corregido;
- nueva build identificada;
- smoke pasa;
- alcance de reejecución definido;
- resultados anteriores marcados como superseded cuando proceda.

21.4. Salida de sprint

- criterios ejecutados;
- S0/S1 = 0;
- S2 corregidos o aceptados;
- suite completa PASS;
- manual PASS;
- build/log si aplica;
- evidencia;
- known issues;
- recomendación de gate;
- documentación.

21.5. Salida de hito

Añade:

- Golden Path;
 - rendimiento;
 - recovery;
 - compatibilidad;
 - accesibilidad;
 - localización;
 - playtest;
 - build candidata;
 - decisión formal.
-

22. Gates de Sprint 16

22.1. Estado final documentado

- compilación: PASS;
- EditMode: PASS;
- PlayMode: PASS;
- prefabs representativos: PASS;
- arquitectura visual, puerta, mobiliario, colliders y navegación: PASS;
- input UI/mundo y click-through: PASS;
- Golden Path post-integración: PASS;
- build Windows x64 externa 0.0.26: PASS;
- guardado, cierre, reapertura y carga equivalente: PASS;
- Player.log: revisado sin errores bloqueantes;
- regresión manual: PASS;
- aprobación visual: PASS;
- aprobación funcional: PASS;
- Sprint 16: COMPLETED / PASS.

22.2. Criterios automáticos

- compilación;
- suite completa;
- tests de StoreInitial;
- referencias;
- no duplicación;
- assets;
- prefabs;
- door behavior;
- wall preference;

- context root;
- input gating;
- save/load;
- smoke de escena.

22.3. Criterios manuales

- muros y escala;
- puerta;
- mobiliario;
- zonas;
- iluminación;
- colliders;
- entrada;
- anchors;
- wall occlusion;
- UI;
- Golden Path;
- load;
- Next Day;
- slots;
- build externa.

22.4. Condición de cierre

Sprint 16 no cierra hasta:

1. aprobar visualmente StoreInitial;
2. confirmar una sola arquitectura;
3. conectar runtime mediante referencias explícitas;
4. corregir propagación de clic;
5. ejecutar regresión;
6. completar Golden Path post-integración;
7. generar build Windows x64 posterior;
8. revisar Player.log;
9. cerrar S0/S1;
10. actualizar evidencia y documentación.

22.5. Signoff de cierre

El Project Owner / VRM Games registra el cierre de Sprint 16 el 2026-07-06 sobre la build 0.0.26. No quedan defectos S0/S1 conocidos dentro del alcance del gate. La evidencia se conserva en los registros de producción, trazabilidad, build y signoff existentes. Este PASS no se propaga automáticamente a Sprint 17 ni a H6.

23. Gates de Sprint 17

23.1. Entrada

- Sprint 16 cerrado;
- build y SHA identificados;
- defects iniciales clasificados;
- escena representativa aprobada;
- Golden Path post-scene PASS.

23.2. Campañas

Balance

- siete días;
- variación de precios;
- stock;
- demanda;
- abandono;
- liquidez.

Rendimiento

- perfil vacío/equipado/carga;
- frame time;
- memoria;
- allocations;
- save/load;
- soak.

Persistencia

- slots;
- backup;
- corrupción;
- recovery;
- equivalencia;
- schema.

UX y accesibilidad

- onboarding;
- Operations;
- input;
- mensajes;
- resolución;
- ES/EN;
- accesibilidad base.

Regresión y build

- suite completa;
- Golden Path;
- build externa;
- logs;
- known issues.

23.3. Salida

- S0/S1 = 0;
- S2 aceptados o corregidos;
- balance documentado;
- objetivos de rendimiento medidos;
- save/recovery PASS;
- UX/accesibilidad PASS;
- build candidata identificada;
- Player.log revisado;
- evidence package;

- recomendación H6.
-

24. Gate H6 — Vertical Slice Approved

24.1. Paquete de evidencia

- build Windows x64;
- versión y SHA;
- hash;
- suites;
- matriz;
- Golden Path;
- siete días;
- performance report;
- save/recovery report;
- UX/accessibility report;
- localization report;
- Player.log;
- capturas/vídeo;
- defects;
- known issues;
- deuda aceptada;
- baseline documental.

24.2. Decisión

PASS Todos los requisitos obligatorios se cumplen.

PASS WITH ACCEPTED DEBT

- no hay S0/S1;
- la deuda no amenaza datos, build o fantasía central;
- owner y plan existen;
- el riesgo está aceptado.

FAIL

- Golden Path imposible;
 - save no confiable;
 - build no reproducible;
 - escena no representativa;
 - rendimiento o estabilidad bloqueante;
 - experiencia incomprensible;
 - evidencia insuficiente para aceptar.
-

25. QA de fases posteriores

25.1. Empleados

Añadir cobertura de:

- contratación;
- salario;
- horarios;

- tareas;
- prioridad;
- navegación;
- bloqueo;
- cancelación;
- descanso;
- cierre;
- save/load;
- competencia por recursos;
- rendimiento con varios agentes;
- explicación UX de inactividad.

25.2. Investigación

- requisitos;
- costes;
- progreso;
- desbloques;
- idempotencia;
- compatibilidad de save;
- UI;
- orden de nodos;
- datos inválidos;
- cambios de catálogo.

25.3. Puestos informáticos

- montaje;
- componentes;
- compatibilidad;
- reserva;
- tarifa;
- uso;
- pago;
- sesión activa al cierre;
- empleados;
- mantenimiento;
- save/load;
- espacio y navegación.

25.4. Comercio online

- inventario compartido;
- reserva;
- picking;
- packing;
- transporte;
- estados;
- cancelación;
- devolución;
- concurrencia;
- capacidad;
- persistencia;
- economía;
- idempotencia de eventos.

25.5. Publishing y desarrollo

- contratos;
 - hitos;
 - pagos;
 - estados;
 - incertidumbre;
 - expiración;
 - cancelación;
 - liquidación;
 - save/migration;
 - balance;
 - UX de riesgo.
-

26. Demo, Alpha, Beta, RC y lanzamiento

26.1. Demo

- recorrido limitado completo;
- first-run;
- reset;
- feedback;
- privacidad;
- hardware variado;
- no exposición de debug;
- known issues adecuados;
- build distribuable;
- save policy clara.

26.2. Alpha

- todas las capacidades comprometidas presentes;
- coverage gap analysis;
- migraciones;
- compatibilidad;
- sesiones largas;
- telemetría autorizada;
- contenido incompleto aceptable;
- triage regular.

26.3. Beta

- feature complete;
- contenido casi completo;
- balance;
- rendimiento;
- drivers;
- localización;
- accesibilidad;
- instalación/actualización;
- crash reporting;
- volumen mayor de usuarios.

26.4. Release Candidate

- regresión completa;

- S0/S1 = 0;
- S2 aceptados;
- requisitos mínimos/recomendados;
- save upgrade;
- instalación;
- desinstalación;
- Steam cuando esté aprobado;
- legal, créditos y privacidad;
- store metadata;
- rollback;
- hash y archivo definitivo.

26.5. Lanzamiento

- smoke del artefacto publicado;
 - descarga e instalación;
 - primera ejecución;
 - actualización;
 - guardados;
 - monitorización;
 - canal de soporte;
 - hotfix plan;
 - clasificación de incidencias;
 - baseline de release congelada.
-

27. QA externo y playtest

27.1. Orden

1. sesiones observadas internas;
2. personas de confianza con build cerrada;
3. playtest estructurado;
4. demo o beta solo con gate aprobado.

27.2. Protocolo

- consentimiento y privacidad;
- versión identificada;
- objetivo de sesión;
- no explicar salvo bloqueo;
- observar;
- registrar tiempo, errores y preguntas;
- encuesta breve posterior;
- separar opinión de defecto;
- evitar prometer funciones futuras.

27.3. Métricas

- tiempo hasta primera acción;
- tiempo hasta primera venta;
- finalización de día;
- uso de ayuda;
- errores de placement;
- confusión entre asignación y reposición;
- abandonos;
- guardado/carga;

- crashes;
 - rendimiento percibido;
 - confianza y comprensión.
-

28. Automatización continua

28.1. Objetivo

Aunque la ejecución local siga siendo válida, la estrategia debe permitir CI cuando el flujo de repositorio lo justifique.

28.2. Pipeline recomendado

```
checkout limpio
→ validar versión de Unity
→ restaurar/cachear Library de forma controlada
→ compilar
→ EditMode
→ PlayMode
→ exportar resultados
→ build opcional por gate
→ archivar logs y artefactos
```

28.3. Requisitos

- mismos packages;
- licencias y credenciales seguras;
- test reports exportados;
- timeout;
- logs;
- fallo del pipeline ante tests fallidos;
- artefactos vinculados al SHA;
- no almacenar secretos en repositorio.

28.4. Flaky tests

- registrar;
- aislar causa;
- no reintentar hasta obtener verde sin investigación;
- permitir un reintento diagnóstico, marcado;
- corregir o poner en cuarentena con owner y fecha;
- mantener cobertura alternativa.

28.5. Cadencia de ejecución

Durante implementación

- compilación continua;
- targeted tests tras cada cambio coherente;
- smoke de escena al modificar composición;
- prueba manual inmediata del comportamiento principal;
- registro de defectos antes de una corrección no trivial.

Antes de integrar

- targeted suite;

- dependencias directas;
- tests nuevos;
- revisión de datos y assets;
- validación de working copy.

Antes de cerrar sprint

- suite completa;
- acceptance matrix;
- manual;
- build si aplica;
- logs;
- known issues;
- evidencia y documentación.

Antes de hito

- ejecución desde checkout/build limpio;
- Golden Path;
- recovery;
- rendimiento;
- compatibilidad;
- accesibilidad/localización;
- playtest;
- decisión formal.

Tras hotfix futuro

- reproducción del defecto;
- fix dirigido;
- regresión de vecinos;
- smoke completo;
- build;
- actualización desde versión anterior;
- comprobación de saves;
- release notes.

28.6. Criterios para automatizar

Automatizar cuando la prueba:

- se repite con frecuencia;
- es determinista;
- protege una invariante crítica;
- tiene alto coste manual;
- cubre muchas combinaciones;
- detecta regresión temprana;
- puede aislarse de presentación cambiante.

Mantener manual cuando:

- el juicio perceptivo es central;
- la UI está cambiando rápidamente;
- el coste de automatización supera el riesgo;
- se trata de exploración;
- el dato esperado no puede formalizarse todavía.

Una prueba manual crítica puede automatizarse después de estabilizar su contrato. La decisión debe considerar mantenimiento, no solo implementación inicial.

29. Evidencias y reporting

29.1. Test Plan de sprint

Debe contener:

- objetivo;
- alcance;
- riesgos;
- entorno;
- suites;
- manual;
- build;
- datos;
- entrada/salida;
- criterios;
- responsables;
- evidencias.

29.2. Acceptance Matrix

Mapea:

criterio → pruebas → evidencia → estado → defecto

No debe limitarse a una lista de criterios sin vínculo a ejecución.

29.3. QA Execution Record

- versión;
- SHA;
- entorno;
- fecha;
- esperado;
- real;
- PASS/FAIL/SKIPPED;
- incidencias;
- defectos;
- recomendación.

29.4. Manual Validation Record

- build;
- checklist;
- resultados;
- capturas/vídeos;
- observaciones;
- incidencias;
- tester.

29.5. Build Execution Record

Debe corregir las carencias observadas en registros históricos e incluir:

- ruta o artefacto;
- tamaño;
- hash;
- versión;
- SHA;
- configuración;
- duración;
- escenas;
- resultado;
- smoke;
- Player.log archivado;
- known issues.

29.6. Informe de hito

- resumen ejecutivo;
 - cobertura;
 - resultados;
 - defectos;
 - rendimiento;
 - compatibilidad;
 - UX;
 - deuda;
 - riesgo;
 - decisión.
-

30. Relación con 08_QA_Testing_Matrix.xlsx

30.1. Función

La matriz será el repositorio operativo de casos y resultados. Este plan define su estructura.

30.2. Hojas recomendadas

1. README
2. Requirements
3. Test_Cases
4. Test_Runs
5. Defects
6. Regression_Sets
7. Environments
8. Builds
9. Coverage
10. Golden_Path
11. Performance
12. Localization_Accessibility
13. Lists

30.3. Campos mínimos de caso

- Test ID;
- Requirement ID;
- área;
- nivel;
- tipo;
- prioridad;

- riesgo;
- título;
- precondiciones;
- datos;
- pasos;
- esperado;
- automatización;
- suite;
- regresión;
- estado de diseño;
- owner;
- notas.

30.4. Campos de ejecución

- Run ID;
- Test ID;
- build;
- versión;
- SHA;
- entorno;
- fecha;
- tester;
- resultado;
- actual;
- evidencia;
- defect ID;
- duración;
- comentarios.

30.5. Identificadores

Formato recomendado:

QA-<AREA>-<NNN>

Áreas:

- BOOT;
- SLOT;
- INP;
- MOV;
- CAM;
- BLD;
- ACC;
- PRD;
- INV;
- ORD;
- RCV;
- DSP;
- CUS;
- RSV;
- QUE;
- CHK;
- DAY;
- ECO;
- SAV;
- UI;
- LOC;
- A11Y;

- ART;
 - AUD;
 - PERF;
 - BUILD;
 - GP.
-

31. Definition of Ready para QA

Una historia, capacidad o sprint está lista para QA cuando:

1. requisito vigente identificado;
 2. criterios definidos;
 3. riesgos identificados;
 4. diseño resuelto;
 5. datos disponibles;
 6. entorno disponible;
 7. build o working copy identificada;
 8. código compila;
 9. tests del desarrollador pasan;
 10. feature flag definido si procede;
 11. persistencia/migración definida;
 12. logs suficientes;
 13. assets y referencias disponibles;
 14. limitaciones documentadas;
 15. matriz preparada.
-

32. Definition of Done de QA

QA se considera cerrado cuando:

1. alcance ejecutado;
 2. criterios trazados;
 3. resultados registrados;
 4. suites completas pasan;
 5. manual requerido pasa;
 6. Golden Path pasa si aplica;
 7. build pasa si aplica;
 8. Player.log revisado;
 9. S0/S1 = 0;
 10. S2 corregidos o aceptados;
 11. regresión ejecutada;
 12. save/load validado para cambios de estado;
 13. rendimiento revisado cuando aplica;
 14. accesibilidad y localización revisadas cuando aplica;
 15. evidencia archivada;
 16. known issues actualizado;
 17. documentación actualizada;
 18. recomendación de gate emitida.
-

33. Catálogo mínimo de campañas

Campaña	Gate	Ejecución
Structural Smoke	cada cambio estructural	automática
Scene Flow	cada sprint relevante	automática + manual
Inventory Integrity	S6 en adelante	automática
Order/Receiving	S7 en adelante	automática + manual
Display/Restock	S8 en adelante	automática + manual
Customer/Shopping	S9-S10 en adelante	automática + PlayMode
Checkout	S11 en adelante	automática + manual
Day Closure	S12 en adelante	automática + manual
Economy	S13 en adelante	automática + simulación
Save/Recovery	S14 en adelante	automática + destructiva controlada
UX/Slots	S15 en adelante	PlayMode + manual
StoreInitial	S16	automática + visual
Stabilization	S17	completa
Golden Path	S16/S17/H6	build
Seven-Day Run	H6	build/manual
Performance	S17/H6	profiler/build
Compatibility	H6 y posteriores	matriz PC
External Playtest	H6 y posteriores	observado

33.1. Checklist maestra de ejecución

La checklist siguiente no sustituye a los casos. Sirve para evitar que una campaña se declare completa dejando una categoría sin revisar.

Identidad

- versión visible correcta;
- SHA registrado;
- working copy limpia o desviaciones documentadas;
- build identificada;
- entorno y hardware registrados;
- fecha y tester registrados.

Compilación y automatización

- Unity abre sin Safe Mode;
- no existen errores de compilación;
- targeted tests ejecutados;
- EditMode completo ejecutado;
- PlayMode completo ejecutado;
- failed/skipped documentados;
- report exportado;
- recuentos esperados comparados con reales.

Escenas y composición

- Bootstrap primero;
- MainMenu;
- StoreInitial;
- ApplicationRoot único;
- contextos registrados;
- roots no duplicados;
- referencias obligatorias;

- fallbacks en estado previsto;
- transición y cleanup.

Gameplay

- movimiento;
- cámara;
- BuildMode;
- pedidos;
- recepción;
- inventario;
- displays;
- clientes;
- reservas;
- cola;
- checkout;
- jornada;
- economía.

Persistencia

- nueva partida;
- guardado;
- segundo guardado;
- backup;
- carga;
- regreso a menú;
- reinicio del proceso;
- recuperación;
- slots independientes;
- borrado;
- equivalencia de estado.

UX

- ratón;
- teclado y foco;
- Escape;
- modales;
- input gating;
- estados vacíos;
- errores;
- tutorial;
- accesibilidad;
- ES/EN;
- resoluciones.

No funcional

- rendimiento;
- memoria;
- tiempos de carga;
- save/load;
- estabilidad prolongada;
- audio/VFX;
- Player.log;
- salida limpia.

Cierre

- defectos registrados;
- severidades revisadas;
- known issues actualizado;
- aceptación trazada;
- capturas/logs archivados;
- recomendación emitida;
- documentos actualizados.

33.2. Auditoría cruzada de invariantes

Antes de aceptar H6 se ejecutará una auditoría específica que compare fuentes de verdad.

Invariante	Comparación
dinero	saldo inicial + movimientos del ledger = saldo final
inventario	suma de ubicaciones y reservas = total lógico
checkout	reservas consumidas = unidades vendidas
pedido	líneas recibidas = unidades creadas, salvo rechazo explícito
displays	stock lógico = representación aprobada
placement	instancias persistidas = ocupación reconstruida
jornada	resultados = transacciones y costes del día
save	estado previo = estado restaurado en campos contractuales
slots	acciones en un slot no modifican los demás
escena	referencias funcionales resuelven sin inferir nombres visuales

La comparación debe definir tolerancias solo donde exista una magnitud no exacta. Dinero, cantidades, IDs y estados no admiten tolerancias aproximadas.

33.3. Muestreo y repetición

Una única ejecución puede no detectar defectos intermitentes. La repetición depende del riesgo:

- transiciones de escena: al menos diez loops en gates relevantes;
- operaciones idempotentes: dos o más ejecuciones consecutivas;
- rutas aleatorias: varias seeds registradas;
- rendimiento: varias ventanas de medición;
- save/load: varios ciclos y reinicio del proceso;
- cierre con agentes: varios estados de cliente;
- input: clic normal, rápido, doble y alternado;
- soak: varias jornadas.

No se fija un número universal. Si aparece un fallo intermitente, se aumenta la repetición hasta comprender su frecuencia y condición.

33.4. Criterios de confianza

La recomendación de QA debe expresar el nivel de confianza:

Confianza	Condición
alta	cobertura completa, evidencia reproducible, resultados estables
media	cobertura suficiente con limitaciones conocidas
baja	huecos importantes, evidencia parcial o entorno no representativo

Un PASS con confianza baja requiere explicar la limitación y normalmente no debe aprobar un hito. La confianza no reemplaza a la severidad; complementa la decisión cuando no es posible probar todas las combinaciones.

33.5. Recomendación final de QA

Toda campaña que aspire a cerrar un sprint o hito termina con una recomendación breve y no ambigua. Debe contener:

- alcance ejecutado y no ejecutado;
- build, versión y SHA;
- resultados automatizados;
- resultados manuales;
- estado del Golden Path;
- defectos abiertos por severidad;
- limitaciones de entorno;
- nivel de confianza;
- deuda aceptada propuesta;
- decisión recomendada.

Formato:

Recomendación: PASS / PASS WITH ACCEPTED DEBT / FAIL

Confianza: alta / media / baja

Bloqueos: ninguno o lista

Limitaciones: lista

Siguiente acción: gate o corrección concreta

QA recomienda; producción acepta o rechaza el gate. Si la decisión final difiere de la recomendación de QA, debe registrarse el motivo, el riesgo asumido y la persona responsable. No se debe modificar retrospectivamente el informe para ocultar esa diferencia.

Una recomendación PASS WITH ACCEPTED DEBT solo es válida si la deuda tiene ID, impacto, workaround, owner, prioridad y condición de pago. La frase “se corregirá más adelante” no es una aceptación suficiente.

34. Mantenimiento del plan

Actualizar este documento cuando:

- cambia el alcance;
- cambia un gate;
- entra una plataforma;
- cambia schema;
- se incorpora un nuevo nivel de prueba;
- una incidencia revela una carencia estratégica;
- cambia la severidad;
- se aprueba hardware mínimo;
- se abre Alpha, Beta o RC;
- se crea la matriz QA.

Los cambios operativos de casos y resultados deben realizarse en la matriz, no inflar este plan con miles de ejecuciones.

Anexo A. Plantillas

A.1. Test Plan

Sprint / Feature Test Plan

Objetivo
Alcance
Fuera de alcance
Riesgos
Entorno
Datos
Automatizado
Manual
Exploratorio
Rendimiento
Build
Entrada
Suspensión / Reanudación
Salida
Evidencias

A.2. QA Execution Record

QA Execution Record

Version:
SHA:
Build:
Environment:
Date:
Tester:

| Suite | Expected | Passed | Failed | Skipped | Result |

Manual
Build
Logs
Defects
Evidence
Recommendation

A.3. Defecto

DEF-XXXX – Título

Severity:
Priority:
Build/SHA:
Environment:
Frequency:

Preconditions
Steps
Actual
Expected
Evidence
Logs
Workaround
Impact
Verification

A.4. Build Record

Build Execution Record

Version:
SHA:
Platform:
Configuration:
Artifact:
Size:
Start/End:
Result:
Player.log:

Preflight
Smoke
Golden Path
Defects
Known Issues

Anexo B. Prácticas históricas conservadas

- prueba automática desde Sprint 0;
- regresión acumulativa;
- scene-flow loops;
- rechazo de transición concurrente;
- aceptación por criterio;
- validación manual de Store;
- build Windows x64;
- ejecución externa;
- registro de incidentes;
- verificación de inventario y acceso;
- recovery y backup;
- slots y autosave;
- recomendación explícita de cierre.

Anexo C. Prácticas reforzadas

- exportar resultados de Test Runner;
- registrar hardware y configuración;
- archivar Player.log;
- guardar hash y tamaño de build;
- distinguir resultado comunicado de evidencia adjunta;
- normalizar estados;
- usar S0-S4;
- mantener trazabilidad requisito-caso-run-defecto;
- medir rendimiento en build;
- conservar datos y seeds de reproducción;
- añadir playtest estructurado antes de hitos públicos.

Anexo E. Ruta de almacenamiento

Documentacion/
└─ 07_QA_Testing_Plan.md

Estado del documento: fuente vigente de estrategia de calidad para la nueva carpeta Documentacion/. La siguiente pieza operativa es 08_QA_Testing_Matrix.xlsx.

Actualización QA W0

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Pruebas de caracterización previas a las olas

Grupo	IDs	Ola protegida	Cobertura mínima
Tiempo y pausa	CHAR-TIM-001 a CHAR-TIM-008	W1	reloj, velocidades, pausa anidada, H
Clientes y checkout	CHAR-CUS-001 a CHAR-CUS-009	W2	ocho activos, checkout obligatorio, F
Pedidos y economía	CHAR-ODR-001 a CHAR-ODR-010	W3	reserva de fondos, Process All, recei
Displays	CHAR-DSP-001 a CHAR-DSP-006	W4	asignación única, cantidades, retorn
Guardado	CHAR-SAV-001 a CHAR-SAV-008	W5	estados seguros, mutaciones pendie
Management	CHAR-MGT-001 a CHAR-MGT-008	W6	día actual + 2, DailySummary, lifetin
Autoría y settings	CHAR-ART-001 a CHAR-ART-007	W7	occlusion false, migración de prefer
Regresión integral	CHAR-REG-001 a CHAR-REG-010	W8	siete días, build externa, logs, save/

Los casos de caracterización fueron ejecutados y validados para cerrar W1-W8 y la Vertical Slice 0.0.26. Cualquier cambio futuro deberá preservar o volver a validar esta cobertura.

Reglas de ejecución y evidencia

- Cada caso se registra primero como EXECUTED / PASS / VALIDATED.
- La caracterización se ejecuta sobre la baseline efbc1a8 antes de modificar la ola protegida.
- Tras implementar una ola se repiten los casos afectados y la regresión dirigida.
- Un resultado debe incluir build/commit, entorno, datos, pasos, expected/actual, log y evidencia.
- Se verifican idempotencia, conservación, ausencia de duplicados y comportamiento tras save/reload.
- W8 incluye campaña de siete días, 720p/1080p, ES/EN, build externa Windows x64 y Player.log.
- Un caso sin evidencia no cuenta como PASS; una prueba documental no sustituye una ejecución.

Regla de cierre y no propagación

- W0-W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.
- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.
-

Regresión adicional S17-MOD-002...010

La candidata posterior a W8 debe ejecutar de nuevo EditMode, PlayMode, Golden Path y build Windows x64. La cobertura adicional incluye visitas con cero producto, un courier por run, no disponibilidad antes del receipt, idempotencia de recepción, LOD persistente, hora HH:mm, reloj en todos los estados, aperturas múltiples, grounding anidado y orientación del staff en rotaciones 0/90/180/270.
